DOM y componentes dinámicos accesibles

Matías Lira Breyan Sebastián

—

Desarrollo Web Profesional

—

José Miguel Carrera Pacheco

DOM VS VIRTUAL DOM

El **DOM (Document Object Model)** es una estructura que representa un documento HTML como un **árbol de objetos** que puede ser manipulado mediante lenguajes como JavaScript.

Cuando una página web se carga, el navegador convierte el HTML en una estructura jerárquica donde cada elemento (div, p, button, etc.) se convierte en un **nodo del árbol DOM**.

Ejemplo de DOM

```
<html>
  <body>
    <h1>Hola</h1>
    <p>Bienvenido</p>
  </body>
</html>
```

Representación en el DOM:

```
html
 └─── body
    ├─── h1
    └─── p
```

Mediante JavaScript se pueden modificar estos elementos para cambiar el contenido o comportamiento de la página.

Virtual DOM

El **Virtual DOM** es una representación **virtual o en memoria del DOM real**.

Este concepto se popularizó en bibliotecas como React desarrollada por Meta Platforms.

En lugar de modificar directamente el DOM real, el sistema:

1. Crea una copia virtual del DOM.
2. Realiza los cambios en esa copia.
3. Compara el nuevo Virtual DOM con el anterior.
4. Actualiza únicamente los elementos que cambiaron en el DOM real.

Este proceso se conoce como diffing.

Diferencias entre DOM y Virtual DOM

Característica	DOM	Virtual DOM
Ubicación	Navegador	Memoria
Actualización	Directa	Mediante comparación
Rendimiento	Puede ser lento si hay muchos cambios	Más eficiente
Uso común	JavaScript tradicional	Frameworks modernos

PROBLEMAS COMUNES AL MANIPULAR EL DOM

Manipular el DOM directamente puede provocar varios problemas, especialmente cuando una aplicación web crece en complejidad.

1. Bajo rendimiento

Cada cambio en el DOM puede provocar recalculación del layout y repintado de la página, lo que afecta el rendimiento si se realizan muchas modificaciones.

Ejemplo:

```
document.getElementById("titulo").innerHTML = "Nuevo texto";
```

Si se hacen cientos de cambios similares, la página puede volverse lenta.

2. Código difícil de mantener

Cuando se manipulan muchos elementos directamente, el código puede volverse complicado y difícil de entender.

Esto ocurre cuando existen muchas funciones modificando distintos elementos de la interfaz.

3. Dependencia del orden del DOM

Si un elemento cambia de posición o estructura, el código JavaScript que lo manipula puede dejar de funcionar correctamente.

4. Problemas de sincronización

Cuando múltiples scripts modifican el DOM, pueden generarse inconsistencias en la interfaz.

GESTIÓN DE FOCO

La gestión de foco se refiere al control del elemento que recibe la interacción del teclado en una página web.

El foco es importante para la accesibilidad, especialmente para usuarios que navegan utilizando el teclado o lectores de pantalla.

Las recomendaciones provienen de las guías de accesibilidad publicadas por el World Wide Web Consortium.

Ejemplo de foco

Cuando se presiona la tecla **Tab**, el navegador mueve el foco entre los elementos interactivos:

- enlaces
- botones
- campos de formulario

Buenas prácticas de gestión de foco

1. Orden lógico de navegación

Los elementos deben seguir un orden coherente al navegar con la tecla **Tab**.

2. Indicadores visuales de foco

El usuario debe poder ver claramente qué elemento tiene el foco.

Ejemplo:

```
button:focus {
outline: 2px solid blue;
}
```

3. Mover el foco cuando cambia la interfaz

Si aparece un diálogo o ventana emergente, el foco debe dirigirse automáticamente hacia ese elemento.

4. Permitir volver al flujo anterior

Cuando se cierra un diálogo, el foco debe regresar al elemento que lo abrió.

USO CORRECTO DE ARIA

ARIA (Accessible Rich Internet Applications) es un conjunto de atributos HTML diseñados para mejorar la accesibilidad de las aplicaciones web dinámicas.

Fue desarrollado por el World Wide Web Consortium dentro de su iniciativa de accesibilidad web.

ARIA permite que tecnologías asistivas como lectores de pantalla comprendan mejor el contenido y la estructura de una página.

Ejemplo básico de ARIA

```
<button aria-label="Cerrar ventana">
X
</button>
```

El atributo aria-label proporciona una descripción que los lectores de pantalla pueden anunciar.

Tipos comunes de atributos ARIA

aria-label

Proporciona una descripción para elementos que no tienen texto visible.

Ejemplo:

```
<button aria-label="Buscar"></button>
```

aria-hidden

Indica que un elemento debe ser ignorado por lectores de pantalla.

Ejemplo:

```
<span aria-hidden="true">*</span>
```

role

Define el tipo de elemento para tecnologías asistivas.

Ejemplo:

```
<div role="alert">
Error en el formulario
</div>
```

Buenas prácticas al usar ARIA

- Usar HTML semántico primero (button, nav, header, etc.).
- Usar ARIA solo cuando sea necesario.
- No reemplazar elementos HTML nativos con ARIA si ya cumplen esa función.